Round 3 — history loaded one row, then on retry loaded many

You have GitHub read access to this repo (`tom2025b/git-vista`, public). Fetch the files named below at their real paths on `main` yourself — do not ask to have them pasted, and do not guess at their contents.

Before answering anything, confirm you can actually read the source: quote the value of `MAX_PAGE_LIMIT` (`crates/git-vista/src/api.rs`). If you cannot find it, say so and stop — a prior round for this exact issue failed silently because its source files uploaded as empty iOS file-picker bookmarks rather than content, and produced a plausible-sounding, worthless answer built from the prompt text alone.

Files to fetch

- `crates/git-vista/src/api.rs` — `fetch_frame`, `fetch_page`, `send_read`, `with_deadline`, `HistoryFetchError`.
- `crates/git-vista/src/app/mod.rs` — where the Frame/page sequence is driven and rendered; look for how a failed or partial load is handled versus a successful one.
- `crates/git-vista-server/src/handlers/read.rs` — the first ~1000 lines (the rest is that file's own test module). `HISTORY_LIMIT` is in `crates/git-vista-server/src/state.rs`.

The bug, in the user's own words

> "at first the history wasn't working. It only showed one line and then after I did it again... it came back with a lot of history."

Also seen in the same session: the banner **"Failed to load history: No active session. Reconnect."** The connection (an SSH local port forward from an iPad) was dropping repeatedly throughout.

Already established — do not re-derive

- History loading is **two-stage**: `fetch_frame()` (refs/metadata, no commits) then `fetch_page()` (one page of rows, paginated by cursor).
- `send_read` (in `api.rs`) already gives reads a bounded attempt plus one retry — this was added to fix a related bug (#218) where reads had neither a timeout nor a retry while writes had both. If you find the retry itself insufficient for this symptom, say precisely why the existing retry doesn't cover the case you're describing, rather than treating its mere presence as the fix.
- `HISTORY_LIMIT` bounds total history server-side; confirm its value and whether it is even reachable in the reported symptom (a single row is far below any plausible limit, so this is likely not about the cap).

What to actually determine

1. **Is "one row" a distinct code path, or is it what a failed/partial load renders as?** Trace: a degraded `Frame` (no `worktree_id`), a `Decode` error on a truncated body, page 1 succeeding but returning genuinely empty, or the Frame succeeding while page 1 fails — which of these, read from the actual code, produces exactly one visible row?
2. **Does a failed load leave stale state a retry then repairs?** Is the row set cleared before a fresh load, or appended to? Could a first, failed attempt leave a single row (e.g. from Frame-derived HEAD-only rendering) that a second, successful attempt then adds onto rather than replaces?
3. **What happens to an in-flight `fetch()` when the underlying TCP connection dies** — which is exactly what an SSH tunnel drop does? Does the retry in `send_read` actually fire for that failure mode, or does the promise hang rather than reject, leaving the retry logic never reached?
4. **The "No active session" banner** — find where it's raised and whether it shares a cause with the truncated-history symptom, or is a separate, unrelated failure that happened to occur in the same session.

Answer format

```
## Verdict
The most likely root cause, stated as a mechanism, one paragraph.

## Mechanism
Step by step, citing file and function.

## Evidence
Quote the actual lines.

## Ruled out
What you eliminated and how.

## Could not determine
Name the exact file/function you'd need next, if any.

## Suggested fix
Concrete, smallest correct change.
```